

Zadanie 1

Napisz klasę **Theatre** opisującą rzędy i miejsca w rzędzie widowni teatru. Dla użytkownika klasy rzędy i miejsca są numerowane od jedynki.

Konstruktor klasy pobiera dowolną liczbę **int**ów oznaczających liczbę miejsc w kolejnych rzędach.

Klasa ma następującą funkcjonalność:

- Metoda **display** drukuje na konsoli wszystkie miejsca zaznaczając które są wolne a które są sprzedane.
- Metoda **buy_ticket** wywołana z dwoma argumentami typu **int**, **row** i **seat**, oznacza kupno biletu na miejsce **seat** w rzędzie **row**.
- Ta sama metoda wywołana z trzema argumentami, **row**, **seat1** i **seat2** oznacza wykupienie biletów na miejsca od **seat1** do **seat2** (włącznie) w rzędzie **row**.
- Metoda **show_free** wywołana z jednym argumentem **row** drukuje listę wolnych miejsc w rzędzie **row**.
- Ta sama metoda wywołana bez argumentów drukuje listy wolnych miejsc we wszystkich rzędach.
- Metoda **show_summary** z ceną jednego biletu jako argumentem drukuje liczbę sprzedanych biletów, liczbę wszystkich miejsc i dochód teatru.

Na przykład program

```
class Theatre:
    #
    # ...
    #

th = Theatre(4, 8, 10, 10)
th.display()
th.buy_ticket(1, 4)
th.buy_ticket(4, 1, 6)
th.buy_ticket(4, 10)
th.buy_ticket(2, 1, 8)
th.buy_ticket(3, 5, 10)
th.display()
th.show_free()
th.show_summary(5)
```

[download Theatre.py](#)

mógłby wydrukować

```
1:          1  2  3  4
2:      1  2  3  4  5  6  7  8
```

```

3:  1  2  3  4  5  6  7  8  9 10
4:  1  2  3  4  5  6  7  8  9 10

```

```

1:           1  2  3 xx
2:      xx xx xx xx xx xx xx xx
3:  1  2  3  4 xx xx xx xx xx xx
4:  xx xx xx xx xx xx  7  8  9 xx

```

```

Row 1, free seats: [1, 2, 3]
Row 2, free seats:  No free seats
Row 3, free seats: [1, 2, 3, 4]
Row 4, free seats: [7, 8, 9]

```

Tickets sold: 22/32, Income: 110

Zadanie 2

Stwórz klasę **Interval** obiekty której reprezentują odcinki $[a, b]$ na osi liczbowej. Końce odcinka, a i b , powinny być zaimplementowane jako „prywatne” własności (*property*) bez „setterów”.

Użyj dekoratora `@functools.total_ordering` dla zaimplementowania wszystkich sześciu operatorów porównania: za „mniejszy” uważamy odcinek o mniejszej współrzędnej a , a jeśli są równe, o mniejszej współrzędnej b .

Jeśli `obj` jest obiektem klasy, to wywołanie `len(obj)` powinno zwracać długość odcinka reprezentowanego przez `obj`.

Wywołanie `obj1.intersect(obj2)` ma zwracać **True** wtedy i tylko wtedy, gdy te dwa odcinki mają niepustą część wspólną.

Wyrażenie `obj1 | obj2` powinno zwracać obiekt reprezentujący najmniejszy odcinek zawierający oba odcinki `obj1` i `obj2`.

Wyrażenie `obj1 & obj2` powinno zwracać odcinek reprezentujący przecięcie odcinków `obj1` i `obj2`, lub powodować wyjątek **ValueError**, gdy przecięcie jest puste.

Zapewnij też rozsądną, krótką reprezentację napisową obiektów klasy.

Na przykład następujący program

```
#!/usr/bin/env python
```

[download Interval.py](#)

```
import functools
```

```
@functools.total_ordering
```

```
class Interval:
```

```
    //
```

```
    // ...
```

```
    //
```

```
i1, i2, i3 = Interval(2, 7), Interval(-4, 4), Interval(-8, 1)
```

```

print('Intervals:', i1, i2, i3)
print('Lengths: ', len(i1), len(i2), len(i3))
print('i1 & i2, i1 & i3, i2 & i2 exist?',
      i1.intersects(i2), i1.intersects(i3), i2.intersects(i3))
print('i1&i2 =', i1 & i2, ' i2&i3 =', i2 & i3)
print('i1|i2 =', i1 | i2, ' i1|i3 =', i1 | i3,
      ' i2|i3 =', i2 | i3)

lst = [i1, i2, i3]
print('Unsorted:', lst)
lst.sort()
print('Sorted: ', lst)

```

powinien wypisać coś w rodzaju

```

Intervals: [2, 7] [-4, 4] [-8, 1]
Lengths:    5 8 9
i1 & i2, i1 & i3, i2 & i2 exist? True False True
i1&i2 = [2, 4]  i2&i3 = [-4, 1]
i1|i2 = [-4, 7] i1|i3 = [-8, 7] i2|i3 = [-8, 4]
Unsorted: [[2, 7], [-4, 4], [-8, 1]]
Sorted:    [[-8, 1], [-4, 4], [2, 7]]

```

Zadanie 3

Stwórz klasę **Peekable**, której konstruktor przyjmuje obiekt iterowalny (*iterable*). Obiekt tej klasy sam ma być iterowalny (czyli mieć metody `__iter__` i `__next__`), a iteracja po nim powinna zwracać elementy przekazanego obiektu iterowalnego. Dodatkowo jednak, metoda **peek** powinna umożliwiać podejrzenie następnego elementu, ale bez jego „konsumowania”.

Na przykład następujący program

```

class Peekable:
    #
    # ...
    #

peekable = Peekable([1, 2, 3, 4, 5])

print(peekable.peek(), end=' ')
print(next(peekable), end=' ')
print(peekable.peek(), end=' ')
print(peekable.peek(), end=' ')
print(next(peekable), end=' ')
print(peekable.peek(), end=' ')
print('\nAnd the rest...')
for e in peekable:
    print(e)

```

[download Peekable.py](#)

powinien wydrukować

```
1 1 2 2 2 3
And the rest...
3
4
5
```

Zadanie 4

Stwórz klasę **Vec2D** reprezentującą wektor w przestrzeni dwuwymiarowej (wektor opisywany jest dwoma współrzędnymi kartezjańskimi v_x i v_y). Za pomocą przeciążania operatorów zdefiniuj operację dodawania

$$[v_{1x}, v_{1y}] + [v_{2x}, v_{2y}] = [v_{1x} + v_{2x}, v_{1y} + v_{2y}]$$

i odejmowania

$$[v_{1x}, v_{1y}] - [v_{2x}, v_{2y}] = [v_{1x} - v_{2x}, v_{1y} - v_{2y}]$$

dwóch wektorów; wynikiem powinien być nowy wektor.

Zdefiniuj też operację mnożenia wektora i liczby,

$$\gamma[v_x, v_y] = [\gamma v_x, \gamma v_y]$$

tak aby działała zarówno wtedy, gdy mnożymy wektor przez liczbę jak i liczbę przez wektor.

Składowe wektora powinny być „prywatne” (w sensie takim jaki jest możliwy w Pythonie). Klasa powinna mieć *read only* własności (*properties*) `vx` i `vy` oraz `magnitude` opisującą długość wektora

$$|\vec{v}| = \sqrt{v_x^2 + v_y^2}$$

Nadpisz też metodę `__repr__` i/lub `__str__`.

Na przykład następujący program

```
va, vb = Vec2D(3, 4), Vec2D(-1, 2)
vc = 2*va - vb*3 + Vec2D(0, 10)
print(va, vb, vc, vc.magnitude)
```

powinien wydrukować

```
V[3, 4] V[-1, 2] V[9, 12] 15.0
```

Zadanie 5

Stwórz menedżer kontekstu **Timer**, który mierzy i drukuje, jak długo wykonuje się blok kodu.

Wywołanie `time.perf_counter_ns()` zwraca bieżący czas z dużą precyzją (w nanosekundach, $1\text{ s} = 10^9\text{ ns}$).

Na przykład program

[download TimerContext.py](#)

```

import time

class Timer:
    #
    # ...
    #

def fibo(n):
    if n < 0: raise ValueError;
    if n <= 1: return n
    return fibo(n-1) + fibo(n-2)

for n in (38, -1, 42, 12, 45, -9, 20):
    print(f'N = {n:2d} ', end='')
    with Timer():
        res = fibo(n)
    if Timer.success: print(f'res={res:>10d}')

```

powinien wydrukować coś w rodzaju

```

N = 38 Time:  2975330663ns res=  39088169
N = -1 ValueError, N negative!  No result
N = 42 Time: 20497105296ns res= 267914296
N = 12 Time:      15362ns res=      144
N = 45 Time: 87347988460ns res=1134903170
N = -9 ValueError, N negative!  No result
N = 20 Time:      532762ns res=      6765

```
